Setup and Execution Guide: KubePilot

1. Prerequisites & Environment

Before deploying or running KubePilot, verify that your host environment meets the following specifications:

Component	Minimum Version	Verified Version	Purpose
Operating System	Windows 10/11, macOS, or Linux	Windows 11 Pro 64-bit	Host environment
Docker Engine	Docker 24.0+	Docker Desktop 28.0+	Minikube container runtime
Minikube	v1.32.0+	v1.34.0+	Local Kubernetes cluster
Kubectl	v1.28.0+	v1.31.0+	Kubernetes CLI
Python	3.11+	Python 3.13.2	Controller & Dashboard runtime
Pip	24.0+	26.1.2	Package manager

2. Cluster Provisioning

Step 2.1: Start Minikube with Docker Driver

Open PowerShell (or Bash) as Administrator and initialize Minikube:

```
# Start Minikube with Docker driver and dedicated resources
minikube start --driver=docker --cpus=4 --memory=6144 --disk-size=20g

# Enable Kubernetes Metrics Server addon
minikube addons enable metrics-server

# Verify cluster connectivity
kubectl cluster-info
kubectl get nodes -o wide
```

Step 2.2: Create Namespace and Deploy Sample Application

Deploy the target namespace and the containerized sample web application:

```
# Create autoscaling demo namespace
kubectl apply -f k8s/namespace.yaml

# Deploy sample web application (Deployment and Service)
kubectl apply -f app/sample-app.yaml

# Verify deployment readiness (Target: 2/2 pods Running and Ready)
kubectl -n autoscaling-demo rollout status deployment/sample-web-app
kubectl -n autoscaling-demo get pods -l app=sample-web-app
```

[!IMPORTANT]

The target deployment **`sample-web-app`** must have at least **2 ready replicas** as mandated by the baseline high availability guardrails.

3. Prometheus Monitoring Setup

KubePilot ingests metrics via standard PromQL queries against Prometheus at **`http://localhost:9090`** (or **`http://prometheus:9090`** in-cluster).

Option A: In-Cluster Prometheus Deployment (Recommended)

Deploy the Prometheus instance into the cluster:

```
# Deploy Prometheus ConfigMap, Deployment, and Service
kubectl apply -f monitoring/prometheus-config.yaml
kubectl apply -f monitoring/prometheus-deployment.yaml

# Wait for Prometheus pod readiness
kubectl -n autoscaling-demo rollout status deployment/prometheus

# Forward Prometheus port to localhost:9090
kubectl -n autoscaling-demo port-forward svc/prometheus 9090:9090
```

Option B: Verification of Prometheus Scraping

In a separate terminal, verify that Prometheus is actively collecting metrics:

```
# Check Prometheus health endpoint
curl -s http://localhost:9090/-/healthy

# Query target deployment container CPU usage
curl -G "http://localhost:9090/api/v1/query" --data-urlencode "query=sum(rate(container_cpu_usa...
```

4. Running the KubePilot Framework

Step 4.1: Install Python Dependencies

Install runtime requirements into your Python environment:

```
# Navigate to project root
cd c:\Users\SRIRAM\design_project

# Install controller and dashboard requirements
pip install -r dashboard/requirements.txt
pip install reportlab pytest pytest-asyncio
```

Step 4.2: Start the KubePilot Web Dashboard & Embedded Controller

The Web Dashboard includes the embedded **`StrategyCoordinator`** daemon which executes the autoscaling reconciliation loop in the background:

```
# Start dashboard on port 5055
python -m dashboard.app
```

- Dashboard UI: **`http://localhost:5055`**
- Health probe: **`http://localhost:5055/healthz`**
- Readiness probe: **`http://localhost:5055/readyz`**

Step 4.3: Running Standalone Controller Daemon (Alternative)

If running in headless or containerized environments without the web interface:

```
# Start standalone reconciliation loop
python -m controller.main
```

5. Executing Benchmark Experiments

KubePilot includes an automated experiment runner that generates synthetic traffic patterns and compares native Kubernetes HPA against the KubePilot PRAH framework.

Running Experiment 1: Stable Workload

Generates a steady 30 requests/second load for 60 seconds across both Baseline HPA and KubePilot:

```
python -m controller.experiment_runner --experiment stable_workload --mode both --duration 60
```

Running Experiment 2: Sudden Traffic Spike

Generates a sudden 120 requests/second load spike for 60 seconds across both Baseline HPA and KubePilot:

```
python -m controller.experiment_runner --experiment traffic_spike --mode both --duration 60
```

Running Both Experiments Consecutively

```
python -m controller.experiment_runner --experiment all --mode both
```

All benchmark measurements are exported to:

- **`experiments/results/exp-<experiment_type>-<timestamp>.csv`**
- **`experiments/results/exp-<experiment_type>-<timestamp>.json`**

6. Verification and Health Checks

Run the following checks to confirm end-to-end operational health:

```
# 1. Verify Kubernetes deployment status
kubectl -n autoscaling-demo get deployment sample-web-app -o jsonpath='{"Desired: "}{.spec.repl...

# 2. Check Dashboard status API
curl -s http://localhost:5055/api/status

# 3. Check live telemetry API
curl -s http://localhost:5055/api/metrics

# 4. Inspect latest autoscaling decisions
curl -s http://localhost:5055/api/decisions

# 5. Verify decision audit trail log file
Get-Content -Tail 5 logs\decisions.jsonl
```

7. Clean Teardown Procedures

To stop KubePilot and teardown the test cluster:

```
# Stop background Python processes
Get-Process python -ErrorAction SilentlyContinue | Where-Object { $_.CommandLine -like "*dashbo...

# Delete deployed Kubernetes resources
kubectl delete -f app/sample-app.yaml
kubectl delete -f monitoring/prometheus-deployment.yaml
kubectl delete -f monitoring/prometheus-config.yaml
kubectl delete -f k8s/namespace.yaml

# Stop Minikube cluster
minikube stop
```

Troubleshooting Guide: KubePilot

1. Kubernetes Connection Failures

Symptoms

- Web dashboard displays **"Kubernetes: Offline"** and status banner reads **"Waiting for Kubernetes response."**
- Target deployment desired and ready replicas show **`Unavailable`**.
- Pod inspection table reports 0 observed pods.
- Exceptions in logs: **`urllib3.exceptions.MaxRetryError`** or **`ConnectionRefusedError: [WinError 10061]`**.

Root Causes

1. Minikube VM or Docker container runtime stopped.
2. Active kubectl context points to a different cluster (e.g., Docker Desktop instead of Minikube).
3. Minikube internal IP address changed upon reboot, causing TLS certificate validation mismatches against **`~/.kube/config`**.

Diagnostic & Recovery Workflow

```
# 1. Check Docker Desktop engine status
docker info

# 2. Check Minikube cluster status
minikube status

# If Minikube is stopped, restart it
minikube start --driver=docker

# 3. Check current kubectl context
kubectl config current-context

# If context is not 'minikube', switch to it
kubectl config use-context minikube

# 4. Verify API server responsiveness
kubectl cluster-info
kubectl get nodes -o wide

# 5. Verify sample deployment pods in target namespace
kubectl -n autoscaling-demo get pods -o wide
```

[!TIP]

The **`KubernetesExecutor`** in KubePilot includes an automatic reconnection routine that periodically attempts to reconnect with exponential backoff without requiring a full dashboard restart.

2. Prometheus Telemetry Offline or Stale Metrics

Symptoms

- Dashboard telemetry indicator shows **"Prometheus: Offline"**.
- CPU and Memory metrics remain frozen at previous values or show default mock values.
- **`/api/metrics`** returns **`"telemetry_source": "mock"`** or **`"is_stale": true`**.

Root Causes

1. The **`kubectl port-forward`** session to the Prometheus service was interrupted or closed.
2. The Prometheus pod in the cluster crashed or was evicted due to memory pressure.
3. Prometheus scraping configuration cannot reach the cAdvisor metrics endpoint on the node.

Diagnostic & Recovery Workflow

```
# 1. Check if Prometheus pod is Running in autoscaling-demo
kubectl -n autoscaling-demo get pods -l app=prometheus

# 2. Inspect Prometheus pod logs for scrape failures
kubectl -n autoscaling-demo logs -l app=prometheus --tail=50

# 3. Restart port-forwarding to localhost:9090
kubectl -n autoscaling-demo port-forward svc/prometheus 9090:9090

# 4. Verify local Prometheus HTTP endpoint responds
curl -s http://localhost:9090/-/healthy

# 5. Run a direct PromQL test query
curl -G "http://localhost:9090/api/v1/query" --data-urlencode "query=up"
```

3. Pods Stuck in Pending or CrashLoopBackOff

Symptoms

- KubePilot executes a scale-up command to 3 or 4 replicas, but ready replicas remain lower than desired replicas.
- **`kubectl get pods`** displays pods in **`Pending`**, **`ContainerCreating`**, or **`CrashLoopBackOff`** state.

Root Causes

1. **Resource Exhaustion:** Minikube node CPU or memory capacity has been exhausted by requested pod resource limits.
2. **Image Pull Errors:** Minikube is unable to pull the container image if local caching or registry access failed.
3. **Probe Failures:** Application startup took longer than the configured **`initialDelaySeconds`** in liveness/readiness probes.

Diagnostic & Recovery Workflow

```
# 1. Identify offending pod names
kubectl -n autoscaling-demo get pods

# 2. Inspect pod scheduling events and failure reasons
kubectl -n autoscaling-demo describe pod <pod-name>

# 3. Check node capacity and allocation
kubectl describe node minikube | Select-String -Pattern "Allocated resources" -Context 0,10

# 4. If resources are saturated, increase Minikube memory/CPUs
minikube stop
minikube start --driver=docker --cpus=4 --memory=6144
```

4. Scaling Blocked by Guardrail Cooldown or Hysteresis

Symptoms

- Workload generates heavy traffic, CPU or latency crosses the policy threshold, but the replica count does not increase.
- Dashboard shows decision action as **"HOLD"** despite active threshold breach.

Root Causes

This is the intended protective behavior of the **Guardrail Engine**:

1. **Scale-Up Cooldown Active:** An earlier scale-up event took place within the last 60 seconds (**`COOLDOWN_SCALE_UP`**).
2. **Scale-Down Cooldown Active:** An earlier scaling action took place within the last 180 seconds (**`COOLDOWN_SCALE_DOWN`**).
3. **Flapping Window Protection:** Directional reversal was detected within the 120-second anti-thrashing window.
4. **Minimum Floor Protection:** Target replica calculation requested *\le 1* replica, which was blocked by the mandatory floor (**`min_replicas: 2`**).

Diagnostic & Verification Workflow

```
# 1. Check latest decision rationale from the API
curl -s http://localhost:5055/api/decisions | jq .

# 2. Inspect decision log for guardrail rejection messages
Get-Content logs\decisions.jsonl -Tail 10

# Expected sample log entry:
# {"action": "HOLD", "reason": "Cooldown active: 24s remaining before scale-up allowed"}
```

5. Port 5055 Socket Conflicts on Windows

Symptoms

- Launching **`python -m dashboard.app`** fails with:

`OSError: [WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted`

Root Cause

An orphaned or background Python process from a previous test run is holding port 5055 open.

Diagnostic & Recovery Workflow

```
# 1. Identify process ID holding port 5055
Get-NetTCPConnection -LocalPort 5055 -ErrorAction SilentlyContinue | Select-Object OwningProces...

# 2. Terminate the blocking process
Stop-Process -Id <PID> -Force

# Or terminate all orphaned dashboard Python processes:
Get-Process python -ErrorAction SilentlyContinue | Where-Object { $_.CommandLine -like "*dashbo...
```

6. HPA and Controller Conflict ("Split-Brain Scaling")

Symptoms

- Replicas oscillate continuously between 1 and 2, or scale-up actions triggered by KubePilot are immediately reverted by native Kubernetes HPA.

Root Cause

Both the native Kubernetes **`HorizontalPodAutoscaler`** resource (**`sample-web-app-hpa`**) and the KubePilot **`StrategyCoordinator`** are attempting to control **`deployment/sample-web-app.spec.replicas`** simultaneously.

Resolution

```
# 1. Inspect existing HPA resources in the namespace
kubectl -n autoscaling-demo get hpa

# 2. If running KubePilot in primary control mode, delete the native HPA:
kubectl -n autoscaling-demo delete hpa baseline-hpa --ignore-not-found

# 3. Verify KubePilot is the sole controller
kubectl -n autoscaling-demo get deployment sample-web-app
```